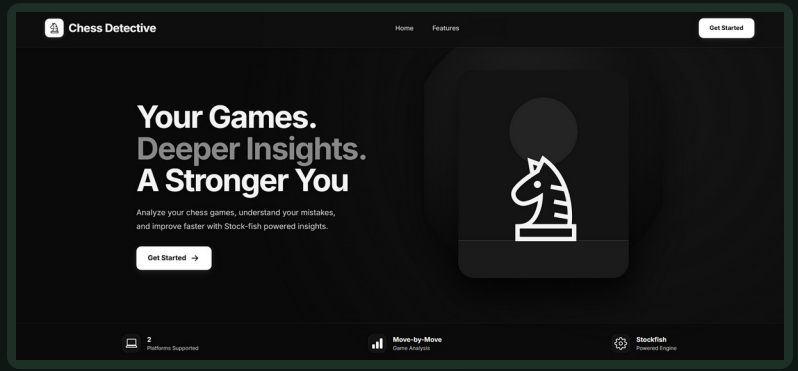
FINAL REVIEW · MINI PROJECT

The Chess Detective

Your games. Deeper insights. A stronger you.

Team: Prabhav Agarwal, Shivansh Gupta
Roll numbers: Prabhav 2500971520139 · Shivansh 2500971520180
Guide: Mr Rup Kamal
Dept: Computer Science & Technology (AI)
Programme: B.Tech. CSE-AI-B, 3rd Semester
Session: 2026–27



Landing page

WHY THIS PROJECT

Problem Statement and Motivation

The problem

Most analysis tools only show what the engine thinks, such as “Stockfish says this move is −1.8”, and leave the player to work out what actually went wrong.

Who is affected

Chess players who play on Chess.com and Lichess and want to understand and learn from their own games.

Why a solution is needed

Players need a clear, move-by-move review: what happened, how good each move was, what the engine preferred, and where the game changed direction.

Goal: investigate the game and understand where things changed, not just see an engine score.

AIM

Objectives and Scope

Objectives

- Fetch games from Chess.com and Lichess by username
- Evaluate every position with Stockfish, locally in the browser
- Classify every move and compute per-side accuracy
- Present an interactive, move-by-move game review

Deliverable: v1.0.0 web app, deployed on Vercel

In scope

- Recent games from both platforms
- Fully client-side analysis
- Opening detection, engine lines, statistics

Boundaries (v1.0.0)

- No accounts or saved game history
- No backend or server-side Stockfish
- No automated test suite yet

CONTEXT

Existing Systems and Research Gap

Chess.com Game Review

- Free, but limited: only one analysis per day
- Detailed review with move classifications
- More analyses need a paid plan

Lichess Analysis

- Free and unlimited analysis
- Results are less accurate
- No detailed move-by-move classification

Chess Detective

- Unlimited analysis, free
- Move-by-move classification of every move
- Engine-backed, win-probability based accuracy
- Runs locally and privately in the browser

Gap: no free tool gives unlimited analysis together with accurate move-by-move classification.

OUR APPROACH

Proposed Solution and Originality

Key features

- Game import from Chess.com and Lichess
- Stockfish analysis in a Web Worker (MultiPV)
- 10 move labels: Book to Blunder, Miss
- Per-side accuracy, openings, engine lines
- Interactive board with eval bar and graph

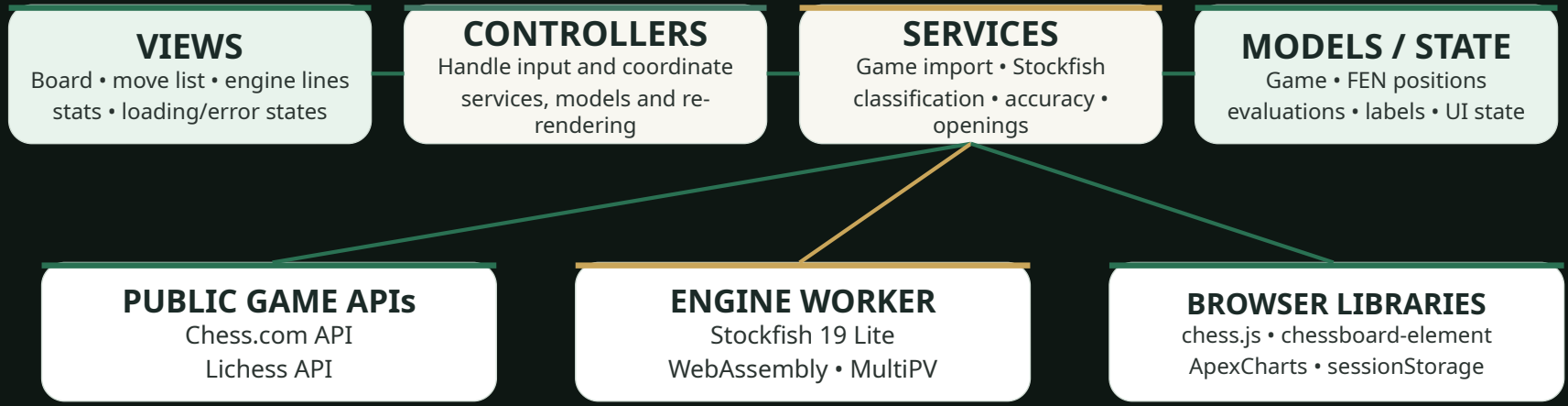
What the team built

- Own independent pipeline for Brilliant and Great moves, built after studying several literatures
- Lichess accuracy model, volatility-weighted
- Chess.com-style classification from expected points
- No backend: private, simple to host

DESIGN

System Architecture

A lightweight MVC-inspired, fully client-side design



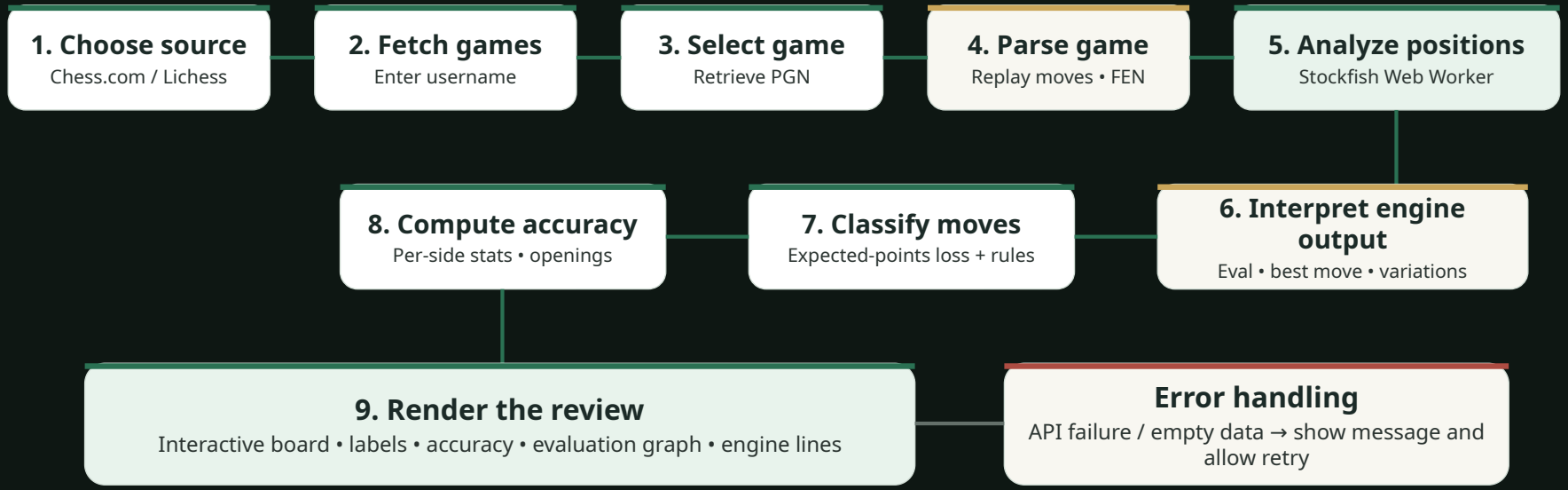
Page flow: index.html → fetchGames.html → gameReview.html

Selected game is transferred with sessionStorage. The current version has no custom backend or database.

METHOD

Methodology and Workflow

From public game retrieval to interactive post-game analysis



The application performs analysis locally in the browser; runtime speed depends on the user's device.

STACK

Technologies and Resources

Frontend	HTML5, CSS3, JavaScript, Vite 5
Chess	chess.js, chessboard-element, Stockfish 19 (lite, WASM), Web Workers
Charts	ApexCharts (evaluation graph)
APIs	Chess.com API, Lichess API
Data	Bundled opening database; sessionStorage for the selected game
Deployment	Vercel (GitHub push triggers build)
Tools	Git, GitHub, Node.js / npm

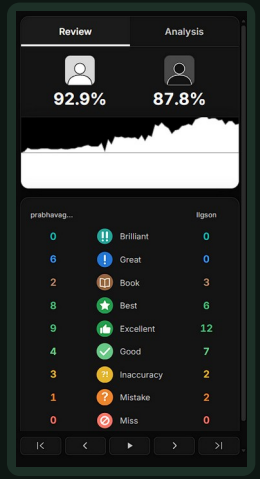
No training dataset or hardware beyond a standard browser is used; the engine and opening data are third-party resources.

BUILD

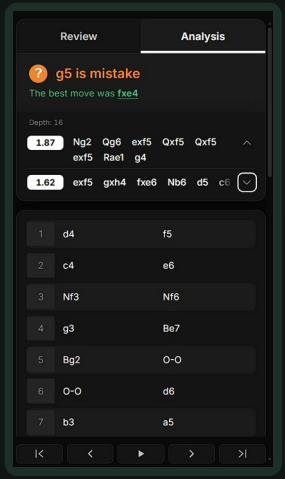
Implementation



Game review dashboard



Move classification

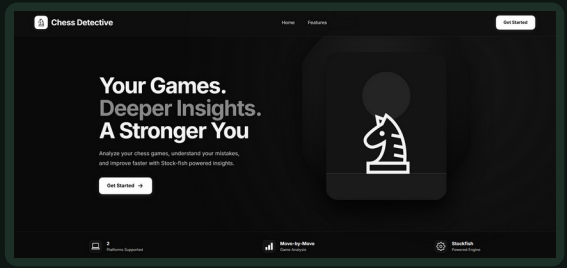


Engine lines

Completed: game import · Stockfish service (Web Worker, MultiPV) · move classification · accuracy and statistics · opening detection · interactive board, eval bar and graph · loading and error overlays

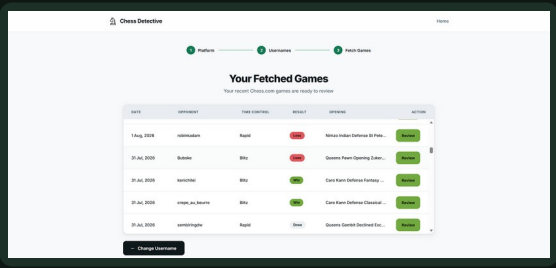
DEMO

Project Demonstration



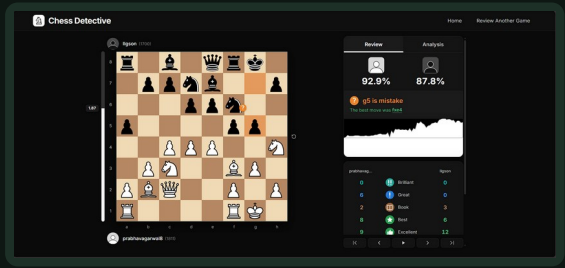
1. Choose

Pick a platform and enter a username



2. Select

Fetch recent games and pick one



3. Review

Stockfish analyses; browse the move-by-move review

Input: platform + username **Processing:** PGN → FEN → Stockfish → classify **Output:** labels, accuracy, graph, lines

Live: chessdetective.vercel.app · Demo video (1:34): youtu.be/bk1xv9qv7NQ

VALIDATION

Testing and Results

Test case	Expected behaviour	Result
Production / Vite preview build	Builds and behaves like the dev version	**Pass**
Invalid or empty game / PGN	Error message shown, no crash	**Pass**
API failure, empty game list	User-facing error state, app stays usable	**Pass**
Stockfish failure or missing lines	Review still loads; missing lines handled	**Pass**
Chess.com month/year rollover	Games from earlier months are fetched	**Pass**
Username or platform change	Fresh game list, no stale data	**Pass**

Objectives achieved

Games fetched from both platforms, every position analysed locally, every move classified, accuracy computed, interactive review working.

Labels by expected-points loss

Best 0 · Excellent ≤ 0.02 · Good ≤ 0.05 · Inaccuracy ≤ 0.10 · Mistake ≤ 0.20 · Blunder > 0.20. Automated tests (Vitest, Playwright) are planned.

LESSONS

Challenges and Solutions

Brilliant and Great moves are complex to categorise	Used AI to understand the calculations, then studied several literatures to build an independent pipeline
Building accuracy, win % and expected-score models	Drafted with Perplexity AI, then implemented the Lichess accuracy model and Chess.com-style classification
Heavy engine search can freeze the UI	Stockfish runs in a Web Worker, with timeouts, worker error handling and progress feedback
API failures and missing data	Defensive handling, fallbacks and user-facing error states

Remaining limitations: analysis speed depends on the device; recent games only; no saved history; no automated tests yet.

TEAM

Team Contributions

Prabhav Agarwal

Roll no.: 2500971520139

Backend: services, models and analysis logic

- Game import service for Chess.com and Lichess APIs, incl. month/year rollover
- PGN parsing and FEN generation before and after every move
- Stockfish service: WASM engine in a Web Worker, MultiPV, timeouts and error handling
- Win % and expected-points model; 10-label move classification
- Lichess-style accuracy, opening detection and API failure fallbacks

Shivansh Gupta

Roll no.: 2500971520180

Frontend: views and controllers

- Landing, game-selection and game-review pages in HTML5, CSS3 and JavaScript, built with Vite 5
- Interactive board (chessboard-element) with move list and navigation
- Evaluation bar and ApexCharts evaluation graph
- Engine lines, accuracy, statistics and move-label panels
- Loading and error overlays; sessionStorage hand-off between pages

Guide: Mr Rup Kamal

WRAP-UP

Conclusion and Future Scope

Achieved

- Working v1.0.0 deployed on Vercel
- Both platforms supported
- Full move classification and accuracy
- Interactive review with opening detection

Limitations

- No accounts or history
- No server-side analysis
- Speed depends on device
- Recent games only
- No automated tests

Next steps

- Deeper tactical detection, better Brilliant/Great
- Node.js backend, accounts, saved analyses
- React rebuild, better mobile UX
- Vitest and Playwright tests

References and Questions

References

[1] The Stockfish Developers, “Stockfish 19,” Stockfish: Strong open source chess engine, 2026. [Online]. Available: https://stockfishchess.org/blog/2026/stockfish-19/

[2] Lichess, “Lichess accuracy metric,” Lichess.org. [Online]. Available: https://lichess.org/page/accuracy

[3] Chess.com, “Published-Data API (PubAPI),” Chess.com. [Online]. Available: https://www.chess.com/news/view/published-data-api

[4] Lichess, “Lichess API documentation,” Lichess.org. [Online]. Available: https://lichess.org/api

[5] J. Hlywa, “chess.js: A JavaScript chess library,” GitHub. [Online]. Available: https://github.com/jhlywa/chess.js

[6] J. Fagnani, “chessboard-element: A custom element for chessboards,” GitHub. [Online]. Available: https://github.com/justinfagnani/chessboard-element

[7] ApexCharts, “ApexCharts.js: Open-source JavaScript charts.” [Online]. Available: https://apexcharts.com

[8] Vite, “Vite: Next generation frontend tooling.” [Online]. Available: https://vite.dev

[9] P. Agarwal and S. Gupta, “Chess Detective: Chess game review web app (v1.0.0),” GitHub, 2026. [Online]. Available: https://github.com/Prabhav-Agarwal/Chess-Detective

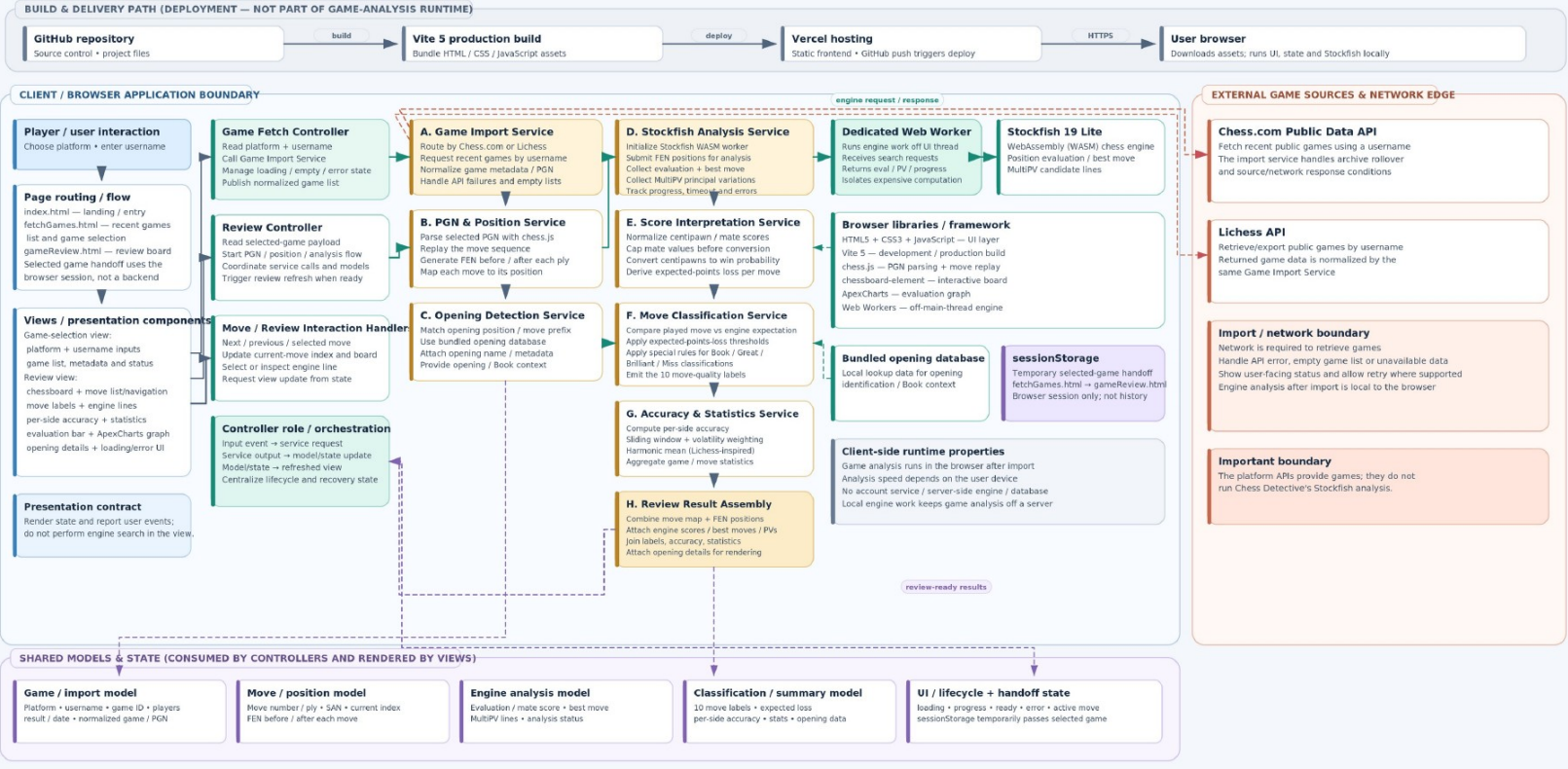
Thank you

Questions and feedback welcome

Prabhav Agarwal (2500971520139)
Shivansh Gupta (2500971520180)
Guide: Mr Rup Kamal

CHESS DETECTIVE — EXTREMELY DETAILED SYSTEM ARCHITECTURE

v1.0.0 • Browser-based chess game review • MVC-inspired frontend • Local Stockfish analysis • Static Vercel deployment



ANALYSIS PIPELINE — DETAILED LOGIC

1 PGN is parsed and replayed; a FEN is generated before / after each ply.
2 Stockfish evaluates positions and returns evaluation, preferred move and MultiPV lines.
3 Mate scores are capped; centipawns are converted to estimated win probability.
4 Expected-points loss is used to classify move quality; special-case rules cover Book, Great, Brilliant and Miss.
5 Per-side accuracy uses a sliding window, volatility weighting and a harmonic mean (Lichess-inspired).
6 Review results join the board position, labels, engine lines, accuracy, opening and statistics.

MOVE QUALITY LABELS & SCORE RULES

Ten labels in the project: Book • Brilliant • Great • Best • Excellent
Good • Inaccuracy • Mistake • Blunder • Miss
Expected-points-loss thresholds documented for standard labels:
Best: 0 | Excellent: ≤ 0.02 | Good: ≤ 0.05
Inaccuracy: ≤ 0.10 | Mistake: ≤ 0.20 | Blunder: > 0.20
Book / Great / Brilliant / Miss use additional special rules.

V1.0.0 BOUNDARIES & RELIABILITY

- No custom backend, login / account system, server-side Stockfish or database.
- No persistent saved-game history; sessionStorage is temporary browser handoff.
- Import scope is recent games from Chess.com and Lichess.
- Device / browser performance affects analysis duration.
- Worker timeouts / errors and API failures are surfaced through loading / error states.
- Missing engine lines are handled; automated test suite is not yet present.

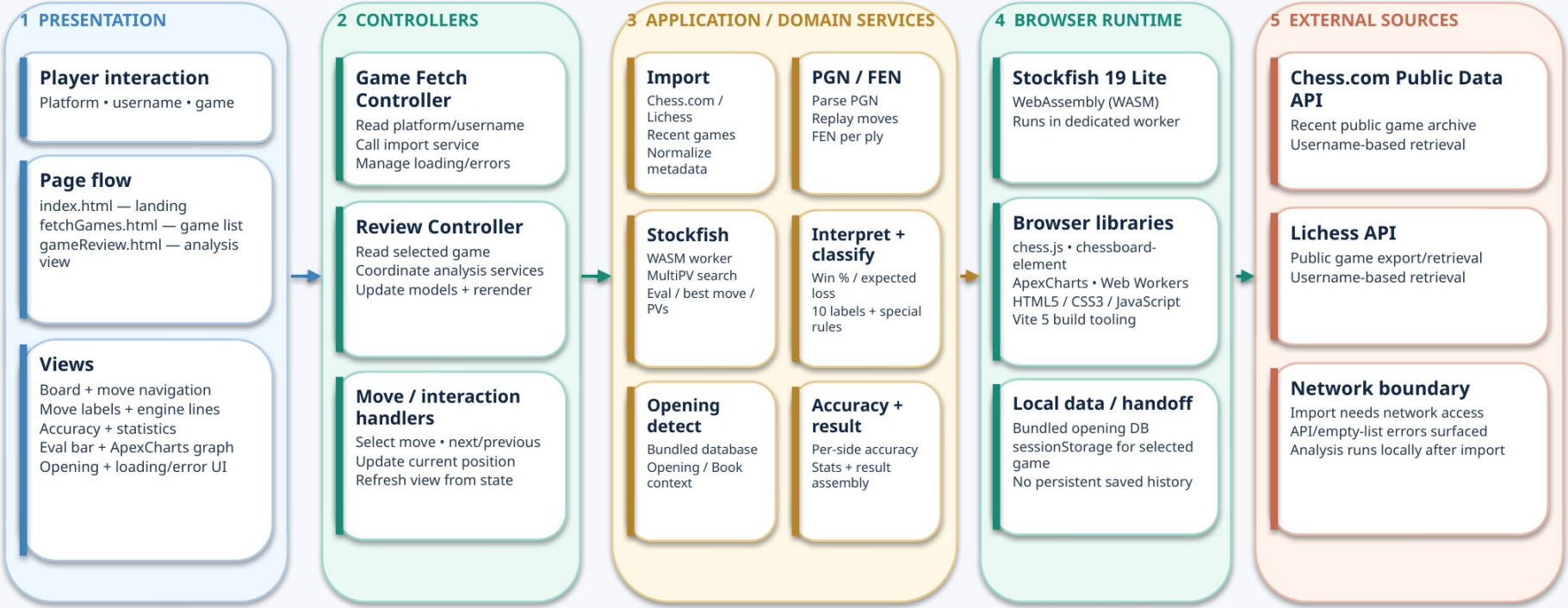
CONNECTOR LEGEND: control / data flow · engine / local library interaction · external API request · state / review result

Source basis: Chess Detective Final Review deck (v1.0.0)

Vector source: Chess_Detective_Detailed_Architecture.svg • Editable component breakdown follows on slides 2–4

Chess Detective | Editable Architecture Overview

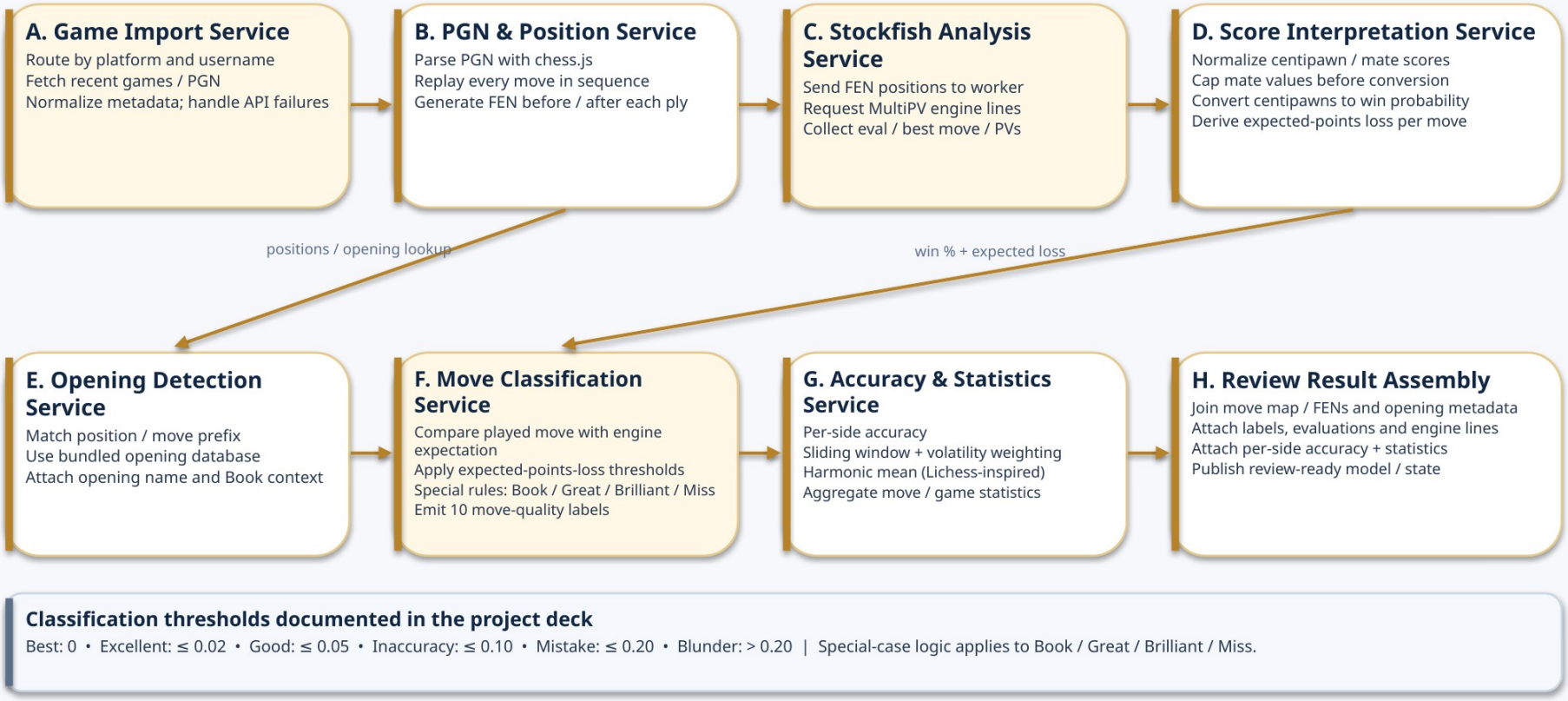
Main runtime components and integration boundaries • v1.0.0



Architecture overview • All service / runtime boxes are editable PowerPoint shapes

Analysis Pipeline | Service-Level Architecture

PGN ingestion → engine evaluation → move interpretation → accuracy / statistics → review-ready output



Classification thresholds documented in the project deck

Best: 0 • Excellent: ≤ 0.02 • Good: ≤ 0.05 • Inaccuracy: ≤ 0.10 • Mistake: ≤ 0.20 • Blunder: > 0.20 | Special-case logic applies to Book / Great / Brilliant / Miss.

Analysis is local to the browser; speed varies by device and browser

Data, Runtime, Reliability & Design Boundaries

What is stored, what runs locally, how failures are surfaced, and what is intentionally out of scope

1 SHARED MODELS / STATE

Game / import model

Platform • username • game ID • players • date • result • PGN

Move / position model

Move number / ply • SAN • current index • FEN before/after each move

Engine analysis model

Evaluation / mate score • best move • MultiPV lines • analysis status

Classification / summary model

10 labels • expected-points loss • per-side accuracy • statistics • opening

UI / lifecycle / transfer state

Loading • progress • ready • error • selected move
sessionStorage transfers selected game from fetchGames.html to gameReview.html
No server-side store or persistent game history

2 BROWSER RUNTIME / LIBRARIES

Web Worker + Stockfish 19 Lite

WASM engine executes off the UI thread
Input: FEN / search requests
Output: evaluation, best move, MultiPV lines and progress
Timeout / worker error handling

Browser-side libraries

chess.js — PGN parsing and move replay
chessboard-element — interactive board
ApexCharts — evaluation graph
HTML5 / CSS3 / JavaScript + Vite 5

Bundled opening data

Opening lookup is local after assets load
Used for opening identification / Book context

Performance boundary

Engine work stays off main UI thread; runtime speed is device-dependent.

3 INTEGRATIONS / FAILURE HANDLING

External game sources

Chess.com Public Data API • Lichess API
Recent public games by username

Import / parse failure paths

API/network error or empty game list
Invalid / malformed PGN or missing game data
Show user-facing error / empty state
Retry where supported; avoid silent failure

Engine failure / partial output

Worker startup / search timeout / worker error
Missing engine lines handled; UI remains responsive
Expose progress / error status as appropriate

V1.0.0 scope limits

No login, backend, server-side engine, database or automated test suite.

BUILD / DEPLOY: GitHub repository → Vite 5 build → Vercel static hosting → browser executes app locally

Source basis: project's Chess Detective Final Review deck, v1.0.0